Phase 8 — Performance Review

AdventureWorks End-to-End BI Project — Execution Plan Analysis & Indexing

Executive Summary

This report analyzes the execution plans of the project's three core incremental-extraction queries — **FactInternetSales**, **DimCustomer** (via its Stored Procedure), and **FactResellerSales** — to identify performance bottlenecks and apply targeted indexing on the AdventureWorks2025 OLTP source. All three queries were tested against the worst-case scenario (watermark = `1900-01-01`, i.e. a full reload), since this is where the highest read volume and query cost occur.

Two recurring bottlenecks were found and addressed with four new non-clustered covering indexes on `ModifiedDate` columns. A third issue — cardinality misestimation on `OR` conditions across joined tables, causing a tempdb spill during sort — was investigated but could not be resolved with statistics updates or query hints, and is documented below as a known limitation rather than a false claim of a fix.

Methodology

- Each query was run in SSMS with `SET STATISTICS IO, TIME ON` and Actual Execution Plan enabled.
- The watermark parameter was set to `1900-01-01` to simulate the worst-case load (a full reload after `FullReloadMode`), which maximizes row counts and query cost.
- For each query, the most expensive operators (by relative cost %) were identified, and an index was proposed only where the root cause was clearly a missing index — not applied speculatively to every table.

Query 1 — FactInternetSales Incremental Extraction

Rows returned: 60,398 | **CPU time:** 1,484 ms | **Elapsed time:** 2,187 ms

Finding 1 — Sort operator dominates the plan (68% of total cost)

The `ROW_NUMBER() OVER (PARTITION BY sod.SalesOrderID ORDER BY sod.SalesOrderDetailID)` expression requires sorting all returned rows, since `Sales.SalesOrderDetail` is clustered by `SalesOrderDetailID` alone, not by `(SalesOrderID, SalesOrderDetailID)`. This Sort operator alone accounted for 68% of the query's total relative cost — by far the single largest cost in the plan.

Finding 2 — Full Clustered Index Scan on both source tables (100% of rows)

Both `Sales.SalesOrderHeader` (31,465 rows) and `Sales.SalesOrderDetail` (121,317 rows) were fully scanned rather than seeked, because neither table had an index on `ModifiedDate` — the column the incremental watermark filter depends on. In a normal daily incremental run (a recent watermark, few

changed rows), this full scan is pure waste: the engine reads the entire table just to filter out almost everything.

Query 2 — DimCustomer Incremental Extraction (ETL.usp_GetDimCustomerIncremental)

CPU time: 312 ms | **Elapsed time:** 310 ms

Finding — Full Clustered Index Scan on Person.Person (31% of total cost)

The single largest cost in this plan was a full scan of `Person.Person` (19,972 of 19,972 rows, 3,817 logical reads — the highest read count in the entire plan), for the same underlying reason: no index exists on its `ModifiedDate` column. The same pattern applies, at lower cost, to `Person.EmailAddress`.

Known limitation — the address-chain CTE is not watermark-filtered

The CTE that resolves each customer's address (joining `BusinessEntityAddress`, `Address`, `StateProvince`, `CountryRegion`, `AddressType`) has no `WHERE` clause of its own — it is fully recomputed for every customer on every run, and the watermark filter is only applied afterwards, in the outer query. This is a structural limitation of the current query design, not something a simple index can fix, and is recorded here as a candidate for a future query redesign rather than addressed in this phase.

Query 3 — FactResellerSales Incremental Extraction

Rows returned: 60,919 | **CPU time:** 1,313 ms | **Elapsed time:** 2,155 ms

Finding 1 — Same missing-index pattern as Query 1

The two largest costs are again full scans of `Sales.SalesOrderDetail` (35% of cost) and `Sales.SalesOrderHeader` (18% of cost) — the exact same root cause as Query 1, since both fact packages read from the same two OLTP tables. No additional indexes were needed beyond the two already proposed for Query 1 — a single pair of indexes benefits both fact-loading packages.

Finding 2 — Cardinality misestimation causing a tempdb spill (investigated, not resolved)

The Sort operator's execution warning reported: *"Operator used tempdb to spill data during execution... Sort wrote 1,088 pages to and read 1,088 pages from tempdb."* The optimizer estimated only 8,431 rows for this operator, versus 60,919 actual rows — a roughly 7.2x under-estimation, forcing the granted memory to be exceeded and the sort to spill to disk.

Root cause: SQL Server's optimizer cannot accurately estimate the selectivity of an `OR` condition spanning two joined tables (`soh.ModifiedDate > @Watermark OR sod.ModifiedDate > @Watermark`), and falls back to a fixed heuristic guess (~27% of the joined row count) regardless of the actual watermark value.

Remediation attempted: `UPDATE STATISTICS ... WITH FULLSCAN` on both source tables, and `OPTION (RECOMPILE)` on the query. Neither changed the estimate or removed the spill warning —

confirming this is an optimizer heuristic limitation, not a stale-statistics problem.

Decision: a full fix (rewriting the filter as a `UNION` of two independently-estimated branches) was evaluated but not implemented, because this cost profile only manifests in the rare full-reload scenario (nearly all rows changed) — not in the daily incremental run this project is designed around, where the actual and estimated row counts are both small and the spill does not occur in practice. This is recorded as a known, accepted limitation rather than fixed, to avoid adding query complexity for a cost that isn't paid in normal operation.

Indexes Created

The following four non-clustered covering indexes were created on `AdventureWorks2025` (the OLTP source) to eliminate the full-table scans identified above. Each is a covering index (via `INCLUDE`) so the engine can satisfy the incremental extraction queries directly from the index, without a Key Lookup back to the base table.

```
CREATE NONCLUSTERED INDEX IX_SalesOrderHeader_ModifiedDate
ON Sales.SalesOrderHeader (ModifiedDate)
INCLUDE (SalesOrderID, SalesOrderNumber, CustomerID, TerritoryID, CurrencyRateID,
OrderDate, DueDate, ShipDate, SubTotal, TaxAmt, Freight);

CREATE NONCLUSTERED INDEX IX_SalesOrderDetail_ModifiedDate
ON Sales.SalesOrderDetail (ModifiedDate)
INCLUDE (SalesOrderID, SalesOrderDetailID, ProductID, SpecialOfferID,
OrderQty, UnitPrice, UnitPriceDiscount, LineTotal);

CREATE NONCLUSTERED INDEX IX_Person_ModifiedDate
ON Person.Person (ModifiedDate)
INCLUDE (BusinessEntityID, FirstName, LastName);

CREATE NONCLUSTERED INDEX IX_EmailAddress_ModifiedDate
ON Person.EmailAddress (ModifiedDate)
INCLUDE (BusinessEntityID, EmailAddress);
```

Index	Table	Benefits
IX_SalesOrderHeader_ModifiedDate	Sales.SalesOrderHeader	FactInternetSales, FactResellerSales
IX_SalesOrderDetail_ModifiedDate	Sales.SalesOrderDetail	FactInternetSales, FactResellerSales
IX_Person_ModifiedDate	Person.Person	DimCustomer
IX_EmailAddress_ModifiedDate	Person.EmailAddress	DimCustomer

Note: the benefit of these indexes is greatest during normal daily incremental runs, where the watermark is recent and only a small fraction of rows have actually changed — that is exactly the scenario where an Index Seek replaces a full-table scan. Because the tests in this report used the worst-case watermark (`1900-01-01`, matching a full reload) to stress-test the plans, the relative improvement from these indexes is expected to be far more dramatic in production-like daily runs than in the full-reload numbers shown here.

Known Limitations (Not Fixed by Design)

- **Cardinality misestimation on cross-table OR conditions** (Query 3, and equally present in Query 1) — occurs only under a full reload; not addressed, as detailed above.
- **Unfiltered address-chain CTE in DimCustomer/DimReseller** — the customer/reseller address resolution recomputes for every row on every run regardless of the watermark; a structural query-design limitation, candidate for a future redesign.

Conclusion

The dominant performance issue across all three queries was the same root cause: the absence of indexes on the `ModifiedDate` columns that the project's watermark-based incremental extraction (Phase 3) depends on. Four targeted covering indexes were added to address this. A secondary, unrelated issue — optimizer cardinality misestimation on OR conditions spanning joined tables — was investigated, found to be an inherent SQL Server optimizer limitation rather than a fixable configuration issue, and is documented as an accepted limitation scoped to the rare full-reload scenario. This phase reflects a deliberate engineering judgment: optimize what recurs in daily operation, and document — rather than over-engineer a fix for — what only affects an infrequent edge case.